



AccumulatR: Simulation and Likelihood Tools for Evidence-Accumulation Models in R

Niek Stevenson
Independent researcher

Abstract

Evidence-accumulation models explain response choices and response times by treating behavior as the result of latent processes that race, combine, or inhibit one another. Applied work often requires more than a single named model family: researchers may want pooled channels, staged onsets, shared absence processes, latent mixtures, or observed responses defined by logical rules. **AccumulatR** is an R/C++ package for specifying, simulating, and evaluating likelihoods for such structured evidence-accumulation models. Model structure is declared in R using accumulators, pools, outcome rules, components, triggers, and mixture settings. The same finalized model object is then used for simulation, data preparation, response-probability calculation, and likelihood evaluation. Repeated numerical work is delegated to compiled code via **Rcpp**. This paper describes the software design, the mathematical representation used by the likelihood evaluator, and a set of reproducible examples covering basic races, pooled outcomes, guarded rules, mixtures, and ranked responses.

Keywords: evidence accumulation, response times, race models, mathematical psychology, R.

1. Introduction: Evidence-accumulation modeling in R

Evidence-accumulation models are a central tool for the analysis of choices and response times in cognitive psychology, neuroscience, and related fields. They replace a purely descriptive account of accuracy and latency with a process account: latent evidence streams evolve over time, an observed response is recorded when one or more streams satisfy a decision rule, and the response time is the finishing time of the relevant process. Diffusion models, linear ballistic accumulator models, lognormal race models, and related dynamic choice models all instantiate this general idea (Ratcliff, Smith, Brown, and McKoon 2016; Brown and Heathcote 2008; Heathcote and Matzke 2022; Rouder, Province, Morey, Gomez, and Heathcote 2015). The software problem is not exhausted by fitting one standard model family. Many sub-

stantive hypotheses are architectural. A task may require multiple routes to a response, a response that depends on a pool of channels, a stop or blocker process, a second stage that can start only after an earlier stage has finished, a mixture of latent processing modes, or data in which the first two responses on a trial are both observed. These requirements are common enough that forcing them into a single winner-takes-all template obscures the model being tested.

Existing software covers important parts of this landscape. Packages such as **RWiener** provide focused implementations for Wiener diffusion models (Wabersich and Vandekerckhove 2014). Recent Stan-based diffusion model tools support flexible Bayesian estimation for particular diffusion parameterizations (Henrich, Hartmann, Pratz, Voss, and Klauer 2024), while broader estimation frameworks such as **lme4** and **brms** provide mature interfaces for hierarchical statistical modeling (Bates, Mächler, Bolker, and Walker 2015; Bürkner 2017). Other work has emphasized hierarchical cognitive models, approximate likelihoods, and joint neural-behavioral models (Vandekerckhove, Tuerlinckx, and Lee 2011; Fengler, Govindarajan, Chen, and Frank 2021; Palestro, Bahg, Sederberg, Lu, Steyvers, and Turner 2018). **AccumulatR** is complementary to these tools. Its main contribution is a direct software representation of structured race-model architectures, coupled to simulation and exact likelihood evaluation.

AccumulatR follows a compact workflow. The user declares the model structure in R, finalizes it, expands a named parameter vector into a trial-wise parameter matrix, and then uses the same finalized model for simulation or likelihood evaluation. The package is implemented as an R package (R Core Team 2025) with computational kernels in C++, exposed through **Rcpp** (Eddelbuettel and Francois 2011). This paper describes both the user interface and the model class it implements. Section 2 introduces the package objects and current public API. Section 3 summarizes the mathematical layer. Section 4 gives the basic R workflow. Section 5 presents structured model examples. Sections 6 and 7 describe likelihood evaluation and implementation details, followed by a comparison to related software and a discussion.

2. Software design

2.1. Objects and workflow

The package distinguishes model structure from numerical parameter values. A `race_spec` object describes the architecture: accumulators, pools, observed outcome rules, components, triggers, mixture settings, and parameter naming rules. A finalized `model_structure` object is produced for simulation, data preparation, response probabilities, and likelihood contexts. The compiled likelihood context is built once and then reused across many parameter evaluations.

The workflow in Table 1 is deliberately explicit. Model-building functions return modified specifications, so they can be chained with the base R pipe. Parameter values are not stored in the model object. Instead `build_param_matrix()` expands a named vector into the rectangular or long parameter layout expected by the simulator and likelihood evaluator. This separation lets the same model architecture be evaluated under many trial-wise parameter sets.

Stage	Main functions	Role
Spec.	<code>race_spec()</code> , <code>add_accumulator()</code> , <code>add_pool()</code> , <code>add_outcome()</code>	Declare the latent accumulators, pooled sources, and observed response labels.
Rules	<code>first_of()</code> , <code>all_of()</code> , <code>none_of()</code> , <code>inhibit()</code> , <code>after()</code> , <code>add_trigger()</code>	Express logical outcome rules, blocking rules, staged onsets, and shared absence events.
Params	<code>set_parameters()</code> , <code>par_names()</code> , <code>build_param_matrix()</code>	Control public parameter names and expand named values to trial-wise parameter rows.
Mixing	<code>add_component()</code> , <code>set_mixture()</code>	Define component membership and specify whether mixture weights are fixed or estimated as sampled-mixture parameters.
Runtime	<code>finalize_model()</code> , <code>simulate()</code> , <code>prepare_data()</code> , <code>make_context()</code> , <code>log_likelihood()</code>	Finalize a specification, simulate data, prepare observations, and evaluate the likelihood.
Inspect	<code>processing_tree()</code> , <code>response_probabilities()</code> , <code>complexity_metrics()</code>	Inspect response logic, marginal response probabilities, and compiled complexity diagnostics.

Table 1: Main public functions in **AccumulatR**.

2.2. Parameter naming

Parameter names are public interface, not an implementation detail. By default, compatible accumulator parameters are grouped. For example, a two-accumulator lognormal race has public parameters `m`, `s`, and `t0`; both accumulators share those values unless the specification requests otherwise. The function `set_parameters()` changes this mapping. Its `separate` argument splits grouped parameters, `share` intentionally maps several internal parameters to one new public name, and `rename` changes a public name without merging distinct parameters. The result can be inspected with `par_names()`.

This design avoids the common ambiguity between an accumulator-level parameter and a public parameter. The name `left.m` is valid only if the specification has separated `m` for the accumulator `left`. Without that declaration, the valid name is `m`. The same mechanism is used for models that combine distribution families. If the same suffix has incompatible meanings across distributions, the default public name is qualified by distribution, for example `lognormal.s` and `RDM.s`.

2.3. Components and mixtures

Components represent qualitatively different process variants within one model. A component declaration defines membership only: it says which accumulators are active in a component, and optionally how many ordered responses that component records. Mixture probabilities

Distribution	Public parameters	Notes
lognormal	<code>m</code> , <code>s</code>	Log-scale location and spread.
gamma	<code>shape</code> , <code>rate</code>	Gamma finishing-time distribution.
exgauss	<code>mu</code> , <code>sigma</code> , <code>tau</code>	Gaussian plus exponential finishing time.
LBA	<code>v</code> , <code>B</code> , <code>A</code> , <code>sv</code>	Linear ballistic accumulator.
RDM	<code>v</code> , <code>B</code> , <code>A</code> , <code>s</code>	Race diffusion model parameterization.

Table 2: Accumulator distributions currently available in **AccumulatR**. All accumulators also have the non-decision-time parameter `t0`, which defaults to zero if omitted.

are configured separately with `set_mixture()`. In `mode = "fixed"`, component weights are supplied as known probabilities. In `mode = "sample"`, non-reference components expose probability parameters such as `p.fast`; the reference receives the residual probability.

The data determine whether a component is observed on a trial. If a `component` column is present, likelihood evaluation conditions on that component. If the column is absent or contains missing component labels, the likelihood marginalizes over components using the model’s mixture specification. The same convention is used by `response_probabilities()`.

3. Model representation

3.1. Leaves, sources, and response rules

The basic latent object is a finishing time. For a leaf accumulator a , let T_a denote its finishing time, with distribution function F_a , density f_a , and survival function S_a :

$$F_a(t) = \Pr(T_a \leq t), \quad f_a(t) = F'_a(t), \quad S_a(t) = 1 - F_a(t). \quad (1)$$

A source is either a leaf or a pool. An outcome rule defines when an observed response becomes available. The simplest outcome rule is a direct readout from one source. More complex rules use first-of, all-of, absence, and inhibition operators.

For a simple independent race in which response r is observed at time t , the familiar density is

$$\ell_r(t) = f_r(t) \prod_{j \neq r} S_j(t). \quad (2)$$

AccumulatR treats Equation 2 as a special case of a more general order-region representation. A target outcome may become true through several transition scenarios, and each scenario is combined with the condition that no competing observed outcome has occurred earlier. For direct, independent races this lowers immediately to products of densities and survival functions. For shared, nested, or guarded rules the evaluator keeps the joint order constraints until they can be projected correctly.

3.2. Pools and logical rules

A pool combines several sources into one source. If pool P has members $T_1, \dots, T_n$ and

finishes when the k th member finishes, then

$$F_P(t) = \Pr \left(\sum_{j=1}^n 1\{T_j \leq t\} \geq k \right). \quad (3)$$

For independent exchangeable members with common F and $S = 1 - F$, this reduces to

$$F_P(t) = \sum_{m=k}^n \binom{n}{m} F(t)^m S(t)^{n-m}. \quad (4)$$

The software does not require exchangeability; the pool is compiled as order constraints over its members.

The helpers `first_of()` and `all_of()` define disjunction and conjunction over response rules. With independent children, `first_of()` becomes true when the earliest child becomes true, and `all_of()` becomes true when the last required child becomes true:

$$f_{\text{first}}(t) = \sum_{r=1}^m f_r(t) \prod_{q \neq r} S_q(t), \quad f_{\text{all}}(t) = \sum_{r=1}^m f_r(t) \prod_{q \neq r} F_q(t). \quad (5)$$

The helper `none_of()` is an absence condition. The helper `inhibit()` defines a response-generating rule and a blocker. For independent direct sources,

$$f_{\text{inhibit}(A,B)}(t) = f_A(t) S_B(t), \quad (6)$$

but the implemented representation also handles composite blockers and shared sources.

3.3. Onsets, triggers, mixtures, and ranks

An accumulator can have an absolute onset or a chained onset. With an absolute delay d_a , the observed finishing time is $T_a = d_a + T_a^*$. With `after()`, the accumulator begins only after another source has finished. If accumulator a starts after source U with lag δ , then

$$T_a = T_U + \delta + T_a^*. \quad (7)$$

The delayed accumulator then participates in the same outcome rules as any other accumulator.

Triggers control whether accumulators are present on a trial. A trigger is a named absence-probability parameter. All members in one `add_trigger()` call share the same absence draw; separate calls create independent absence draws. If a shared trigger has absence probability q_g , the trial likelihood is a mixture over the trigger-on and trigger-off states:

$$L = (1 - q_g) L^{\text{on}} + q_g L^{\text{off}}. \quad (8)$$

Mixture components use the same likelihood principle. If a component is observed, the trial likelihood conditions on that component. If it is latent, the likelihood averages over components:

$$L_i = \sum_c \pi_{ic} L_{ic}. \quad (9)$$

For ranked-response data, the model can retain more than one ordered response per trial by setting `n_outcomes` in `race_spec()`. The likelihood is then evaluated sequentially over the observed response-time pairs $(r_1, t_1), \dots, (r_K, t_K)$.

4. Using AccumulatR

4.1. A two-choice race

The following example defines a two-choice lognormal race. Before parameter separation, both accumulators share the default public parameters:

```
R> library("AccumulatR")
R> spec <- race_spec() |>
+   add_accumulator("left", "lognormal") |>
+   add_accumulator("right", "lognormal") |>
+   add_outcome("left", "left") |>
+   add_outcome("right", "right")
R> par_names(spec)

[1] "m"  "s"  "t0"
```

To give the two accumulators separate lognormal locations and spreads, the specification uses `set_parameters()`. The non-decision time `t0` remains shared and is not supplied below, so it defaults to zero.

```
R> model <- spec |>
+   set_parameters(separate = list(m = TRUE, s = TRUE)) |>
+   finalize_model()
R> pars <- c(
+   left.m = log(0.28), left.s = 0.16,
+   right.m = log(0.35), right.s = 0.18
+ )
R> param_df <- build_param_matrix(model, pars, n_trials = 120)
R> sim <- simulate(model, param_df, seed = 20260612)
R> head(sim, 4)
```

	trials	R	rt
1	1	right	0.3167664
2	2	left	0.2847782
3	3	left	0.3259237
4	4	left	0.2515444

The simulated data use the same columns expected for real behavioral data: `trials`, `R`, and `rt`. The likelihood workflow prepares the data once, builds a compiled context once, and evaluates parameter matrices as needed.

```
R> prepared <- prepare_data(model, sim[c("trials", "R", "rt")])
R> ctx <- make_context(model)
R> log_likelihood(ctx, prepared,
+   build_param_matrix(model, pars, trial_df = prepared))

[1] 157.0651
```

Response probabilities can be computed without simulation:

```
R> one_trial <- build_param_matrix(model, pars, n_trials = 1)
R> round(response_probabilities(model, one_trial), 3)

  left right
0.823 0.177
```

The value returned by `log_likelihood()` is a density-based log-likelihood, so it need not be negative. With continuous response-time models, densities can be greater than one.

4.2. Trial-wise parameters

The parameter matrix is trial-wise by construction. A typical fitting function therefore takes a low-dimensional vector of free parameters, maps it to the public parameter names of the model, and then calls `build_param_matrix()` with the prepared data. For example, a minimal negative log-likelihood function for the two-choice race is:

```
neg_loglik <- function(theta) {
  est <- c(
    left.m = theta[["left.m"]],
    left.s = exp(theta[["log_left.s"]]),
    right.m = theta[["right.m"]],
    right.s = exp(theta[["log_right.s"]])
  )
  params <- build_param_matrix(model, est, trial_df = prepared)
  -as.numeric(log_likelihood(ctx, prepared, params))
}
```

The package does not impose an optimizer or sampler. The likelihood can be used with `optim()`, custom maximum-likelihood routines, or external Bayesian engines that call back into R.

5. Structured model examples

5.1. Pooled evidence

Pools represent responses generated by a threshold number of latent channels. In the following specification, response A is observed when any two of three A channels have finished, while response B is generated by a single competing accumulator.

```
pool_model <- race_spec() |>
  add_accumulator("A1", "lognormal") |>
  add_accumulator("A2", "lognormal") |>
  add_accumulator("A3", "lognormal") |>
  add_accumulator("B", "lognormal") |>
  add_pool("A_pool", c("A1", "A2", "A3"), k = 2L) |>
  add_outcome("A", "A_pool") |>
  add_outcome("B", "B") |>
  set_parameters(separate = list(m = TRUE, s = TRUE)) |>
  finalize_model()
```

When A is observed, its response time is the finishing time of the second finisher in `A_pool`. The likelihood uses that pooled completion rule directly; the user does not need to derive a separate density for the order statistic.

5.2. Logical response rules

Observed responses can be defined by logical rules rather than by single accumulators. The next model says that a response occurs only if an encoding stage has finished, either choice route has finished, and the stop process has not occurred.

```
logical_model <- race_spec() |>
  add_accumulator("encoding", "lognormal") |>
  add_accumulator("left_route", "lognormal") |>
  add_accumulator("right_route", "lognormal") |>
  add_accumulator("stop", "lognormal") |>
  add_outcome(
    "choice",
    all_of(
      "encoding",
      first_of("left_route", "right_route"),
      none_of("stop")
    )
  ) |>
  finalize_model()
```

For stop-like architectures where one process generates a response and another can block it, `inhibit()` gives a more direct expression:

```
stop_model <- race_spec() |>
  add_accumulator("go", "lognormal") |>
  add_accumulator("stop", "lognormal") |>
  add_outcome("respond", inhibit("go", "stop")) |>
  finalize_model()
```

Both examples use string labels directly. The helper `build_outcome_expr()` is available for programmatic builders that need quoted R expressions.

5.3. Chained onsets and shared triggers

Chained onsets express staged processing. Here accumulator C starts only after latent accumulator B has finished:

```
onset_model <- race_spec() |>
  add_accumulator("A", "lognormal") |>
  add_accumulator("B", "lognormal") |>
  add_accumulator("C", "lognormal", onset = after("B")) |>
  add_outcome("A", "A") |>
  add_outcome("C", "C") |>
  set_parameters(separate = list(m = TRUE, s = TRUE)) |>
  finalize_model()
```

Triggers express shared absence events. In the next model, the two go accumulators share one absence draw and one absence-probability parameter:

```
trigger_model <- race_spec() |>
  add_accumulator("go1", "lognormal") |>
  add_accumulator("go2", "lognormal") |>
  add_outcome("R1", "go1") |>
  add_outcome("R2", "go2") |>
  add_trigger("shared_trigger", members = c("go1", "go2")) |>
  set_parameters(separate = list(m = TRUE, s = TRUE)) |>
  finalize_model()
```

If separate `add_trigger()` calls are used instead, the absence draws are independent. Their probabilities can still be given a shared public parameter through `set_parameters()`; that changes parameter naming but not the independence of the trigger draws.

5.4. Latent mixtures

Components are useful when trials may come from qualitatively different processing modes. The following model has a fast and a slow target route, with one competitor. The component declarations define which accumulators are active in each component, and `set_mixture()` declares the component weights to be sampled parameters.

```
mix_model <- race_spec() |>
  add_accumulator("target_fast", "lognormal") |>
  add_accumulator("target_slow", "lognormal") |>
  add_accumulator("competitor", "lognormal") |>
  add_pool("target", c("target_fast", "target_slow")) |>
  add_outcome("R1", "target") |>
  add_outcome("R2", "competitor") |>
  add_component("fast", members = c("target_fast", "competitor")) |>
  add_component("slow", members = c("target_slow", "competitor")) |>
  set_parameters(separate = list(m = TRUE, s = TRUE)) |>
  set_mixture(mode = "sample", reference = "slow") |>
  finalize_model()
```

For this model, the public parameter `p.fast` is the probability of the fast component and the slow component receives the residual mass. If the data include a `component` column, the likelihood conditions on the observed component. Without that column, `fast` and `slow` are treated as latent processing modes and marginalized.

5.5. Ranked responses

Some experiments record more than one ordered response on a trial. Setting `n_outcomes = 2L` asks the simulator and likelihood evaluator to retain the first and second finishing responses:

```
rank_model <- race_spec(n_outcomes = 2L) |>
  add_accumulator("A", "lognormal") |>
  add_accumulator("B", "lognormal") |>
  add_outcome("A", "A") |>
  add_outcome("B", "B") |>
  set_parameters(separate = list(m = TRUE, s = TRUE)) |>
  finalize_model()
```

The corresponding data columns are `R`, `rt`, `R2`, and `rt2`. Higher ranks follow the same naming pattern.

6. Likelihood evaluation and fitting

The likelihood interface is intentionally narrow. `prepare_data()` checks and reshapes behavioral observations. `make_context()` compiles the model-side runtime state. The likelihood call combines a context, prepared data, and a parameter matrix. This separation matters for estimation because model compilation is usually more expensive than evaluating one new parameter matrix.

```
prepared <- prepare_data(model, data_df)
ctx <- make_context(model)

objective <- function(theta) {
  values <- transform_theta_to_named_parameters(theta)
  params <- build_param_matrix(model, values, trial_df = prepared)
  -as.numeric(log_likelihood(ctx, prepared, params))
}

fit <- optim(start, objective, method = "Nelder-Mead")
```

The transformation placeholder in this example is user code: it maps unconstrained optimization variables to the public parameter names of the model. For example, positive spread, boundary, and non-decision-time parameters are commonly optimized on the log scale and exponentiated before calling `build_param_matrix()`. This keeps the package focused on model evaluation while leaving estimation strategy to the analysis.

The parameter matrix may be rectangular, with one row per accumulator per trial, or long, with rows aligned to prepared data for component-specific architectures. The `layout = "auto"` default selects the appropriate layout for most uses. Prepared data can also be compressed when repeated observations share the same prepared representation; in that case the likelihood sums weighted compact rows and expands trial-wise values when requested.

7. Implementation

The implementation has three layers. The first layer is the R specification interface. It validates names, stores accumulator definitions, records outcome expressions, normalizes parameter mappings, and finalizes a model into a runtime-oriented structure. The second layer prepares data and parameters for repeated evaluation. The third layer is the compiled evaluator, which receives the finalized model plan and performs simulation or likelihood calculations. The exact likelihood evaluator lowers outcome rules into transition scenarios. A scenario records which latent source releases a target rule at the observed time, which other sources must already be available, and which blockers or competitors must not have occurred earlier. These statements are represented as order constraints over latent finishing times. The compiler simplifies contradictory or redundant cells, projects simple constraints to density, distribution, or survival factors, and retains numerical integration only where the order structure cannot be reduced further.

This representation is important for models with shared sources. If two observed outcomes both depend on a common latent stage, they can tie with positive density because the same latent finishing time releases both rules. Treating every tie as a zero-probability boundary would be incorrect. By keeping shared latent identity in the region representation, **AccumulatR** can distinguish ordinary zero-measure equality from model-induced equality.

Most computational cost that depends only on model structure is paid when the context is built. Repeated likelihood calls reuse the compiled plan and mostly pay the cost of evaluating densities, probabilities, and remaining integrals under new parameter values. For complicated architectures the number of symbolic cells can grow quickly. When a context is built with `diagnostics = TRUE`, `complexity_metrics()` reports the compiled complexity so the user can inspect whether a specification is becoming computationally expensive.

8. Relation to existing software

AccumulatR is not a replacement for specialized diffusion-model packages or for full estimation systems. Its role is narrower and more structural. A package such as **RWiener** provides direct density, distribution, and random generation functions for the Wiener first-passage-time model (Wabersich and Vandekerckhove 2014). Stan-based tools provide Bayesian fitting workflows for particular diffusion parameterizations (Henrich *et al.* 2024). General modeling systems such as **lme4** and **brms** focus on regression and hierarchical estimation interfaces (Bates *et al.* 2015; Bürkner 2017). Cognitive modeling frameworks such as those discussed by Vandekerckhove *et al.* (2011) address hierarchical sequential-sampling models from an estimation perspective.

The distinguishing feature of **AccumulatR** is that the race architecture itself is the central software object. The same object is used for simulation, inspection, response-probability

calculation, and likelihood evaluation. This is useful when the scientific question is whether a response is generated by a pool, a staged route, a latent mixture, or a guarded process, rather than only which parameter values best fit a fixed named model.

There are corresponding limitations. The package currently assumes that leaf accumulators are conditionally independent given parameters, components, and trigger states. It supplies likelihood evaluation but not a complete hierarchical modeling language. Users who need group-level priors, posterior sampling diagnostics, or formula-based regression over parameters must supply that estimation layer separately. In exchange, they obtain a model object whose architecture can be manipulated directly in R.

9. Replication material

The manuscript directory contains executable R scripts under `dev/LatexFiles/examples`. The script `01-basic-workflow.R` reproduces the two-choice race in Section 4, including the reported response probabilities. The script `02-structured-models.R` constructs pooled, logical, trigger, onset, and ranked-response examples. The script `03-mixtures-and-fitting.R` constructs the sampled-mixture example and runs a small likelihood-based estimate of the mixture weight. The top-level `code.R` script sources the examples and prints `sessionInfo()`, which is the form expected for JSS replication material.

10. Discussion

AccumulatR provides a declarative interface for a class of structured evidence-accumulation models. Its core design choice is to make architecture explicit: accumulators, pools, outcome rules, triggers, components, and ranked readouts are part of the model object rather than ad hoc code around a fixed density. The same object then supports simulation, response-probability calculation, data preparation, and likelihood evaluation.

This architecture-first design is most useful for analyses where the main scientific hypothesis concerns process structure. It also keeps the package honest about what it does not do. **AccumulatR** evaluates models; it does not attempt to be a universal optimizer, sampler, or hierarchical modeling system. That boundary keeps the interface small and makes it easier to combine the package with external estimation tools.

The most important future work is broader empirical demonstration and a richer estimation interface around the existing likelihood engine. Additional finishing-time families, more diagnostic summaries, and tighter integration with hierarchical workflows would also be natural extensions. The present software already supplies the central machinery needed for those extensions: a compact model language for structured accumulator architectures and a reusable compiled evaluator for the likelihoods implied by those architectures.

Acknowledgments

The package and manuscript benefited from the published literature on sequential-sampling models and from the *Journal of Statistical Software* style and replication guidelines.

References

- Bates D, Mächler M, Bolker B, Walker S (2015). “Fitting Linear Mixed-Effects Models Using Lme4.” *Journal of Statistical Software*, **67**, 1–48. ISSN 1548-7660. doi:[10.18637/jss.v067.i01](https://doi.org/10.18637/jss.v067.i01).
- Brown SD, Heathcote A (2008). “The Simplest Complete Model of Choice Response Time: Linear Ballistic Accumulation.” *Cognitive Psychology*, **57**(3), 153–178. ISSN 0010-0285. doi:[10.1016/j.cogpsych.2007.12.002](https://doi.org/10.1016/j.cogpsych.2007.12.002).
- Bürkner PC (2017). “Brms: An R Package for Bayesian Multilevel Models Using Stan.” *Journal of Statistical Software*, **80**, 1–28. ISSN 1548-7660. doi:[10.18637/jss.v080.i01](https://doi.org/10.18637/jss.v080.i01).
- Eddelbuettel D, Francois R (2011). “**Rcpp**: Seamless R and C++ Integration.” *Journal of Statistical Software*, **40**(8), 1–18. doi:[10.18637/jss.v040.i08](https://doi.org/10.18637/jss.v040.i08).
- Fengler A, Govindarajan LN, Chen T, Frank MJ (2021). “Likelihood Approximation Networks (LANs) for Fast Inference of Simulation Models in Cognitive Neuroscience.” *eLife*, **10**, e65074. ISSN 2050-084X. doi:[10.7554/eLife.65074](https://doi.org/10.7554/eLife.65074).
- Heathcote A, Matzke D (2022). “Winner Takes All! What Are Race Models, and Why and How Should Psychologists Use Them?” *Current Directions in Psychological Science*, **31**(5), 383–394. ISSN 0963-7214. doi:[10.1177/09637214221095852](https://doi.org/10.1177/09637214221095852).
- Henrich F, Hartmann R, Pratz V, Voss A, Klauer KC (2024). “The Seven-parameter Diffusion Model: An Implementation in Stan for Bayesian Analyses.” *Behavior Research Methods*, **56**(4), 3102–3116. ISSN 1554-3528. doi:[10.3758/s13428-023-02179-1](https://doi.org/10.3758/s13428-023-02179-1).
- Palestro JJ, Bahg G, Sederberg PB, Lu ZL, Steyvers M, Turner BM (2018). “A Tutorial on Joint Models of Neural and Behavioral Measures of Cognition.” *Journal of Mathematical Psychology*, **84**, 20–48. ISSN 0022-2496. doi:[10.1016/j.jmp.2018.03.003](https://doi.org/10.1016/j.jmp.2018.03.003).
- Ratcliff R, Smith PL, Brown SD, McKoon G (2016). “Diffusion Decision Model: Current Issues and History.” *Trends in cognitive sciences*, **20**(4), 260–281. ISSN 1364-6613. doi:[10.1016/j.tics.2016.01.007](https://doi.org/10.1016/j.tics.2016.01.007).
- R Core Team (2025). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL <https://www.R-project.org/>.
- Rouder JN, Province JM, Morey RD, Gomez P, Heathcote A (2015). “The Lognormal Race: A Cognitive-Process Model of Choice and Latency with Desirable Psychometric Properties.” *Psychometrika*, **80**(2), 491–513. ISSN 1860-0980. doi:[10.1007/s11336-013-9396-3](https://doi.org/10.1007/s11336-013-9396-3).
- Vandekerckhove J, Tuerlinckx F, Lee MD (2011). “Hierarchical Diffusion Models for Two-Choice Response Times.” *Psychological Methods*, **16**(1), 44–62. ISSN 1939-1463. doi:[10.1037/a0021765](https://doi.org/10.1037/a0021765).
- Wabersich D, Vandekerckhove J (2014). “The RWiener Package: An R Package Providing Distribution Functions for the Wiener Diffusion Model.” *The R Journal*, **6**(1), 49–56. ISSN 2073-4859.

Affiliation:

Niek Stevenson

Independent researcher

E-mail: niek.stevenson@gmail.com

URL: <https://github.com/niekstevenson/AccumulatR/>